

Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación



Proyecto 03

Elección de tecnologías de base de datos

CC3089 - Base de Datos 2
Semestre I 2026

Roberto Barreda, 23354
Javier España, 23361
Diego López, 23747
Ángel Esquit, 23221
Mia Fuentes, 23775
Anggelie Velásquez, 221181

Guatemala, 28 de mayo de 2026

Índice

1. Introducción	2
2. Objetivo	2
3. Tecnología elegida	2
4. Modelo de datos	2
4.1. Capturas del grafo en Neo4j	3
5. Importación de archivos	5
5.1. Formato esperado	5
6. Algoritmo de resolución	5
6.1. Salida esperada	6
6.2. Evidencia del armado	6
7. Ejemplos de resultados	7
8. Conclusiones	9
9. Referencias	9

1. Introducción

Este proyecto aborda la representación y resolución de rompecabezas físicos mediante una base de datos de grafos. La idea central es modelar cada rompecabezas como un conjunto de piezas con conexiones numeradas, de modo que el programa pueda consultar relaciones entre piezas y generar instrucciones de armado a partir de una pieza inicial.

La solución está pensada para ser escalable: permite importar diferentes rompecabezas, registrar piezas faltantes, persistir ese estado y reutilizar el mismo modelo para topologías distintas, como cadenas lineales, cuadrículas o estructuras irregulares.

2. Objetivo

El objetivo es seleccionar una tecnología de base de datos adecuada, justificar la decisión y construir una solución funcional que:

- almacene la información de los rompecabezas y sus piezas;
- represente las relaciones físicas entre conexiones;
- permita importar archivos de texto con el formato del proyecto;
- resuelva el rompecabezas partiendo de una pieza inicial;
- indique qué hacer cuando existen piezas faltantes.

3. Tecnología elegida

Se eligió Neo4j como base de datos principal. La razón es que el problema se modela naturalmente como un grafo: cada pieza es un nodo y cada unión entre conexiones es una relación. Este enfoque reduce la complejidad de consultar vecinos y recorrer la estructura del rompecabezas.

Entre las ventajas más importantes están:

- consultas directas sobre vecinos de una pieza;
- facilidad para representar topologías distintas sin alterar el esquema;
- persistencia simple del estado de disponibilidad de las piezas;
- buena correspondencia con el algoritmo de recorrido por anchura usado en la solución.

4. Modelo de datos

El modelo implementado organiza la información en tres niveles principales:

- **Puzzle:** almacena información general del rompecabezas, como identificador, nombre, marca, material, temática y total de piezas.
- **Pieza:** representa cada pieza individual y guarda un campo **disponible** para indicar si la pieza está presente o faltante.
- **Conex:** representa cada punto de unión físico de una pieza, con número y tipo (*MACHO* o *HEMBRA*).

Las relaciones principales son:

- (Puzzle)-[:TIENE]->(Pieza) para agrupar piezas por rompecabezas;
- (Pieza)-[:TIENE_CONEXION]->(Conex) para registrar las conexiones de cada pieza;
- (Conex)-[:ENLAZA]->(Conex) para modelar la unión física entre dos conexiones compatibles.

4.1. Capturas del grafo en Neo4j

Las siguientes imágenes muestran el grafo resultante en Neo4j para distintos rompecabezas y estados de carga.

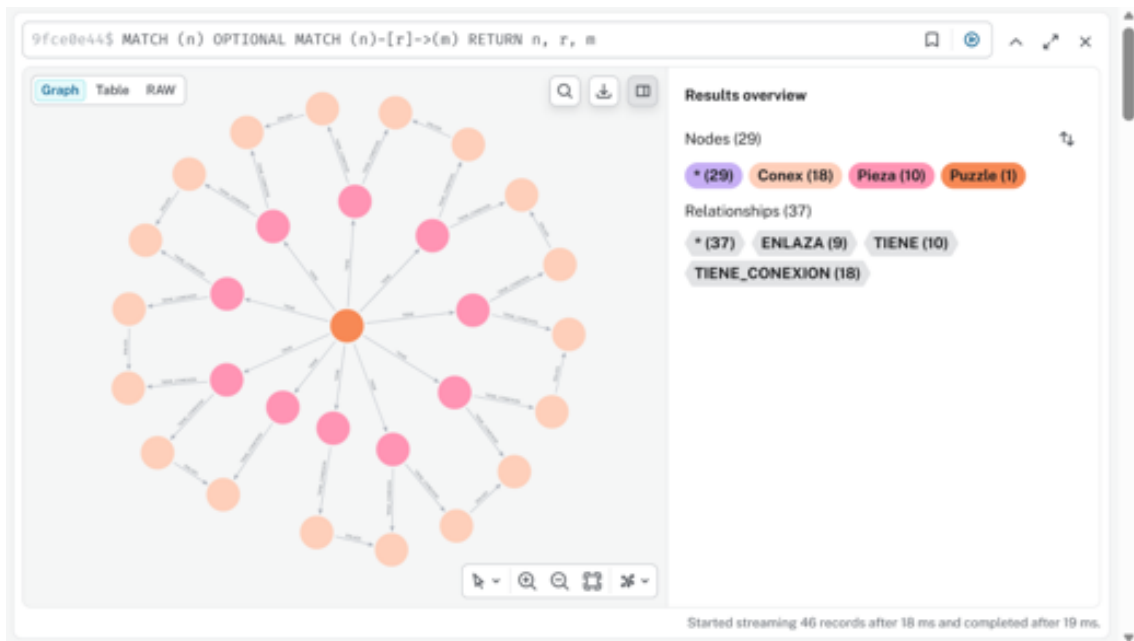


Figura 1: Grafo del rompecabezas Plesiosaur completamente armado desde la pieza inicial.

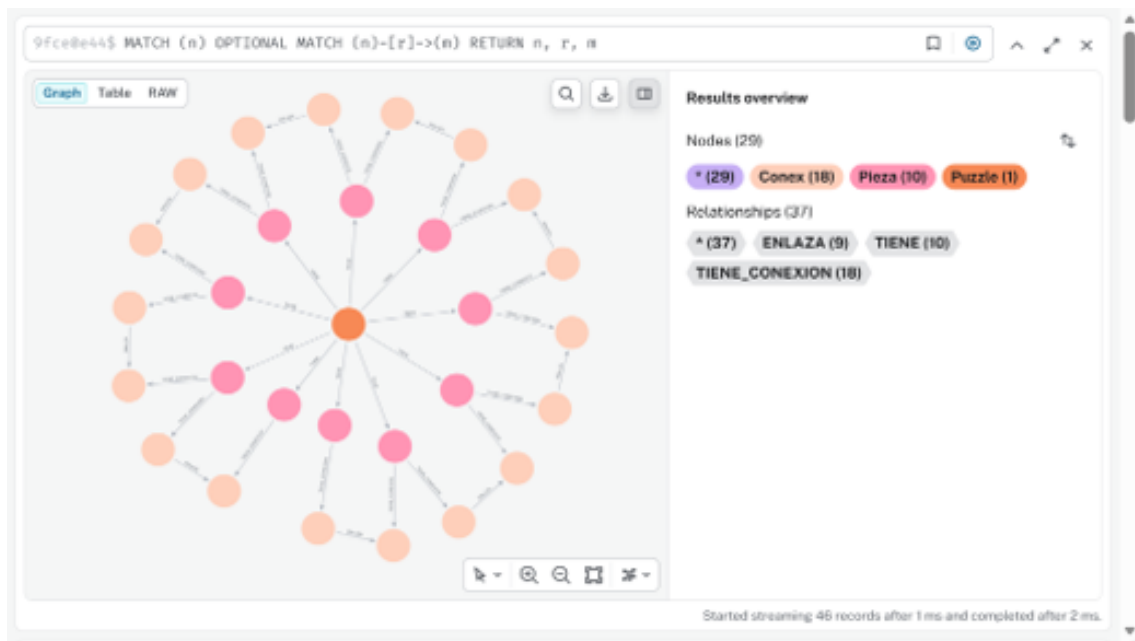


Figura 2: Grafo del rompecabezas Plesiosaur con una pieza faltante registrada en la base de datos.

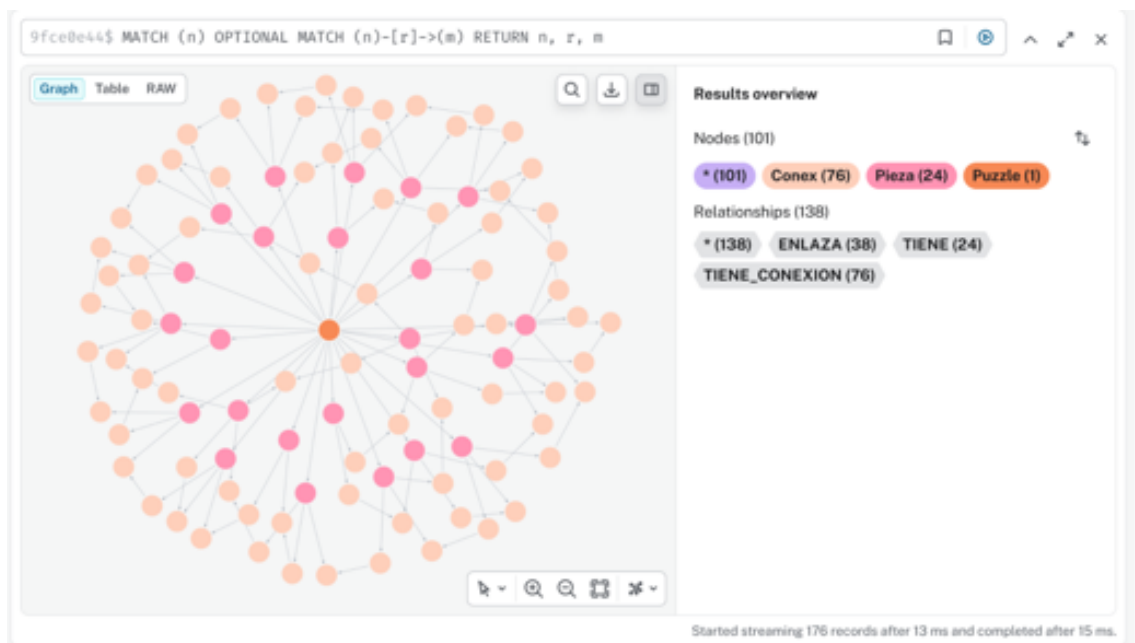


Figura 3: Grafo del rompecabezas Aviones resuelto completamente desde la pieza R1C1.

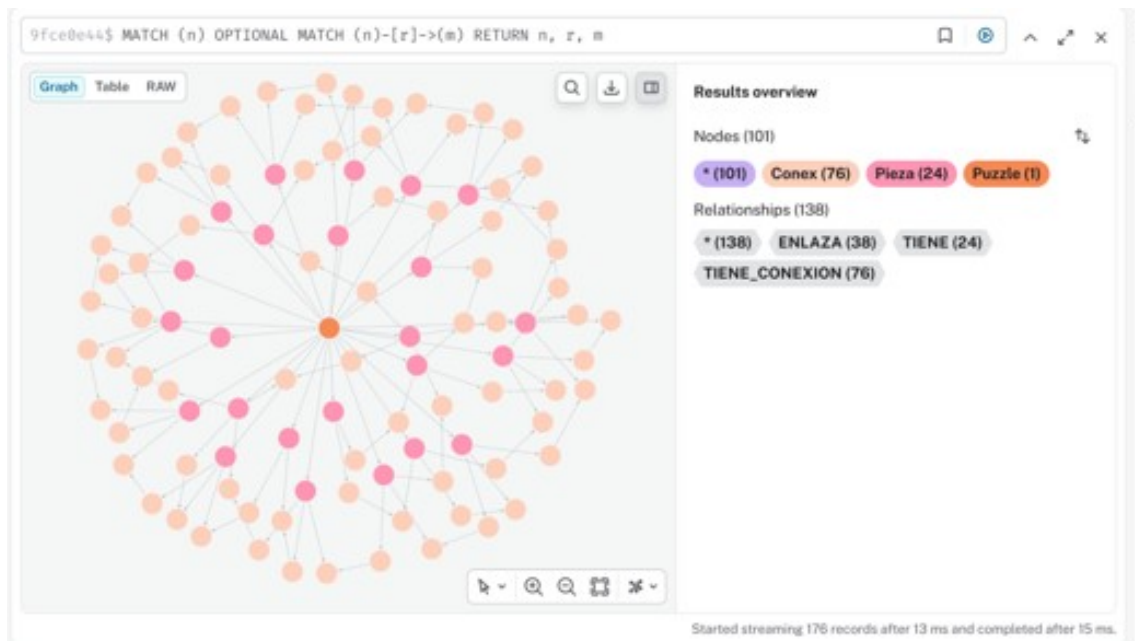


Figura 4: Grafo del rompecabezas Aviones en estado parcial, con tres piezas faltantes.

5. Importación de archivos

Los rompecabezas se cargan desde archivos de texto con secciones definidas para el rompecabezas, piezas, conexiones y enlaces. Durante la importación, el parser valida la estructura, la existencia de campos obligatorios y la consistencia de conexiones y enlaces.

Además de evitar errores de formato, esta validación protege la base de datos de datos inconsistentes. Si un archivo contiene piezas duplicadas, conexiones inexistentes o un enlace entre dos conexiones del mismo tipo, la importación se detiene con un error explicativo.

5.1. Formato esperado

El archivo debe contener, al menos:

- [PUZZLE] con `puzzle_id` y `nombre`;
- [PIEZAS] con una lista de `labels`;
- [CONEXIONES] con `pieza|numero|tipo`;
- [ENLACES] con `piezaA|conexionA|piezaB|conexionB`.

6. Algoritmo de resolución

La solución usa un recorrido por anchura (BFS) sobre el grafo de piezas y conexiones. A partir de una pieza inicial, el programa consulta sus vecinos y va generando instrucciones de armado. Si una pieza vecina está marcada como faltante, el algoritmo no la usa para continuar el recorrido, pero sí la reporta como pieza ausente.

El comportamiento más importante es que, cuando se termina un grupo conectado de piezas disponibles, el algoritmo no se detiene. En su lugar, el programa salta a otra pieza disponible no visitada y continúa el recorrido. Esto permite resolver varios mini-rompecabezas independientes

dentro del mismo tablero lógico, que es precisamente el caso que puede ocurrir cuando las piezas faltantes rompen la conectividad.

6.1. Salida esperada

El resultado final incluye:

- instrucciones paso a paso;
- orden de armado;
- piezas faltantes detectadas;
- piezas no alcanzadas desde los componentes procesados.

6.2. Evidencia del armado

Las fotos del armado físico muestran la correspondencia entre el modelo lógico y el rompecabezas real.



Figura 5: Vista superior del armado físico del rompecabezas Plesiosaur.



Figura 6: Vista lateral del armado físico del rompecabezas de aviones.

7. Ejemplos de resultados

En la práctica, el sistema permite resolver diferentes rompecabezas con estructuras distintas. En las capturas del grafo se observa cómo la solución se adapta a cada topología, manteniendo el mismo esquema de almacenamiento.



Figura 7: Rompecabezas Plesiosaur resuelto físicamente.



Figura 8: Rompecabezas de aviones resuelto físicamente.

8. Conclusiones

Neo4j resultó una opción adecuada para este proyecto porque el problema está dominado por relaciones entre piezas y no por operaciones tabulares tradicionales. La solución permite importar, persistir, consultar y resolver rompecabezas de forma consistente.

Además, el manejo de piezas faltantes no solo reporta la ausencia, sino que conserva la capacidad de resolver todo lo que sí está conectado. Esto mejora la utilidad práctica del sistema y se alinea con la observación explícita de la consigna.

9. Referencias

- Consigna del Proyecto 03 - Elección de tecnologías de base de datos.
- Neo4j Aura: <https://neo4j.com/cloud/aura/>.
- Documentación del proyecto y archivos fuente incluidos en el repositorio.